

ECE 5760 Lab 1: Hardware ODE solver with HPS control

Md Shad, Michael Grillo, Stephen Barnett

February 13, 2026

1 Introduction

In this first lab, we implemented a hardware-based ordinary differential equation (ODE) solver on the DE1-SoC FPGA to integrate the Lorenz system utilizing a Digital Differential Analyzer (DDA). The solver we implemented was a first-order Euler integration method which updates the system's three coupled state variables, which are $x(t)$, $y(t)$, and $z(t)$. This is done in fixed-point arithmetic, allowing us to do a real-time computation of the chaotic Lorenz attractor. We created a design where you can configure parameter values and initial conditions so an user can explore and test the system's sensitivity to small perturbations and highlighting the concept of deterministic yet unpredictable behavior.

The DDA core was implemented in synthesizable Verilog and executed entirely in FPGA hardware, while the Hard Processor System (HPS) provided high-level control and visualization. The HPS communicated with the solver through memory-mapped parallel I/O (PIO) interfaces, issuing commands to start, pause, reset, and adjust integration parameters. Solver outputs were streamed to the HPS and rendered on a VGA display, which plotted all three two-dimensional projections of the Lorenz attractor in real time.

We also had a command-line interface running on the HPS, which allows users to dynamically modify the Lorenz parameters (σ , β , and ρ), integration timestep, and initial conditions during execution. The system also supported real-time control of animation speed with speed up and slow down features, pausing and resuming integration, and clearing the display without breaking the computation. Through careful fixed-point design, hardware–software co-design, and verification using ModelSim and SignalTap, the final implementation demonstrated accurate numerical integration, responsive visuals, and efficient and accurate coordination between FPGA logic and the HPS.

2 Mathematical Background

2.1 Lorenz System

The Lorenz system consists of three nonlinear ordinary differential equations.

$$\frac{dx}{dt} = \sigma(y - x) \quad (1)$$

$$\frac{dy}{dt} = x(\rho - z) - y \quad (2)$$

$$\frac{dz}{dt} = xy - \beta z \quad (3)$$

The Lorenz system defined above follows the formulation presented in the ECE 5760 laboratory handout [1].

The variables $x(t)$, $y(t)$, and $z(t)$ help describe and define the state of the Lorenz system at time t . The constants σ , ρ , and β control the behavior of the system.

Each parameter has a physical and mathematical meaning within the Lorenz model:

σ The parameter σ controls how strongly x is driven toward y . It determines the rate at which the horizontal velocity difference equalizes. Larger values cause x to respond more quickly to changes in y .

ρ The parameter ρ represents the temperature difference driving convection and acts as the main energy input to the system. When ρ is small the motion settles to a steady state, while larger values can produce oscillatory and eventually chaotic behavior.

β The parameter β is a damping term that limits growth in the z direction. It models vertical heat loss and prevents the system from diverging to infinity.

The initial conditions in our design, $x(0)$, $y(0)$, and $z(0)$ are the starting point of the trajectory in state space.

Because the Lorenz system is chaotic, very small changes in these starting values produce completely different long-term trajectories, even though the governing equations remain the same.

The timestep Δt determines how far forward in time the Euler integrator advances on each iteration. A smaller timestep improves numerical accuracy but requires more iterations, while a larger timestep runs faster but can introduce numerical error or instability.

2.2 Discrete-Time Update Equations

The FPGA operates in discrete clock cycles, so we had to make sure the continuous-time equations must be approximated using numerical integration. We used the forward Euler method, which updates the system by making a small step in time at each iteration.

At time step k , the current state of the system is (x_k, y_k, z_k) . The next state is computed using

$$x_{k+1} = x_k + \Delta t \cdot \sigma(y_k - x_k) \quad (4)$$

$$y_{k+1} = y_k + \Delta t \cdot (x_k(\rho - z_k) - y_k) \quad (5)$$

$$z_{k+1} = z_k + \Delta t \cdot (x_k y_k - \beta z_k) \quad (6)$$

In hardware, the FPGA uses the current values of x , y , and z , multiplies them by the timestep Δt , and adds the result onto the previous state. Each clocked update therefore advances the simulation forward by a small amount of time.

2.3 Properties of the Lorenz System

For these parameter values the Lorenz system displays chaotic behavior. The equations are deterministic, meaning the next state is completely determined by the current state, yet long-term prediction becomes impossible because very small differences in the starting conditions grow rapidly over time.

Instead of settling to a constant value or repeating pattern, the trajectory remains bounded while continually moving through space. The resulting shape is known as a **strange attractor** and appears as a butterfly-like structure when plotted.

This property makes the Lorenz system a useful test of a numerical solver. Small numerical errors from fixed-point rounding or timing differences quickly change the trajectory, so correct operation of the hardware solver can be verified visually from the plotted curve.

Because the Euler method is a first-order numerical approximation, smaller timesteps improve stability and prevent divergence of the solution in fixed-point arithmetic.

3 Roles of the ARM (HPS) and FPGA

The design used a hardware–software co-design approach in which the FPGA executed the real-time computation while the ARM-based Hard Processor System (HPS) managed control and communication. The FPGA was assigned to deterministic, high-throughput numerical computation, while the ARM processor provided system-level control, user interaction, data management, and graphics rendering. This separation allowed the Lorenz system to be integrated in real time without burdening the processor with strict timing constraints.

FPGA Responsibilities The FPGA implemented the core numerical solver using a Digital Differential Analyzer. At each integration step, the FPGA computed the Lorenz system derivatives and updated the state variables using a first-order Euler method. All arithmetic was performed using signed fixed-point representations to efficiently map onto the Cyclone V DSP blocks and to ensure predictable latency. The use of hardware multipliers enabled parallel evaluation of terms such as xy , $(\rho - z)x$, and $\sigma(y - x)$, which would be comparatively expensive in software.

By executing the integration loop entirely in hardware, the FPGA provided deterministic timing behavior and consistent numerical update rates independent of processor load. This was especially important for chaotic systems such as the Lorenz attractor, where small timing variations or dropped updates could significantly alter the resulting trajectory. The solver advanced only when explicitly clocked or enabled, allowing precise synchronization with the visualization and user commands.

The FPGA also implemented memory-mapped parallel I/O (PIO) registers that served as the interface to the HPS. These registers exposed the current state variables (x , y , and z) and accepted configuration values including the Lorenz parameters (σ , β , ρ), the integration

timestep, and control signals such as reset and run/stop. Fixed-point results were sign-extended in hardware to 32-bit values to simplify software-side access and maintain numerical correctness.

ARM (HPS) Responsibilities The ARM processor running on the HPS acted as the system orchestrator. Its primary role was to configure and supervise the hardware solver rather than perform numerical integration itself. Through memory-mapped I/O writes, the ARM initialized the solver state, updated parameters, and controlled the execution flow of the DDA. This allowed the user to modify parameters dynamically, reset the system, or pause and resume integration without reprogramming the FPGA.

In addition to control, the ARM was responsible for data consumption and visualization. It periodically read the solver outputs from the FPGA and mapped the numerical results to screen coordinates for display. The ARM handled all VGA drawing operations, including plotting the three orthogonal projections of the Lorenz attractor, clearing the screen, and overlaying textual information such as parameter values and system status. Offloading graphics to the processor avoided unnecessary complexity in the FPGA design while leveraging the flexibility of software for rendering and user interface tasks.

The ARM also implemented a command-line interface that allowed interactive control of the system at runtime. User commands were parsed in software and translated into appropriate hardware register updates, enabling real-time experimentation with different parameter sets and initial conditions. This interface provided a convenient and extensible mechanism for interacting with the solver without introducing additional hardware inputs.

System-Level Integration The separation between computation and control resulted in a clean and scalable architecture. The FPGA operated as a deterministic numerical engine, while the ARM provided adaptability, debugging capability, and user-facing features. Communication between the two domains occurred exclusively through well-defined memory-mapped registers, which simplified verification and debugging. Hardware correctness was verified independently in ModelSim and SignalTap, while software behavior could be modified and tested without altering the hardware design.

Overall, this division of labor allowed the system to combine the performance and predictability of hardware with the flexibility and ease of development offered by software. The resulting design demonstrates an effective approach to real-time numerical simulation on an FPGA-based system-on-chip platform.

4 RTL Design

This design implements one update of the Lorenz system across three separate dimensions (x , y , z) each clock cycle, computed through the use of 7.20 fixed-point arithmetic. We chose to use this resolution of fixed point to balance the range of possible values with maximum precision. When we cleverly scale values mid computation, we end up with integrated values not exceeding $|64|$, allowing us to store signed numbers within this range, leaving the other 20 bits for as high resolution as possible in fixed point, while leaving each 27 bits, the largest

multiply value within the DSP blocks on the board. We chose to use fixed-point as fixed-point allowed for a very simple way to store values and to update values that just required an initial scale by 2^{20} rather than the significantly more complex hardware needed to support floating-point arithmetic, which for a Lorenz system would produce the same macroscopic output. This RTL design followed a similar format to the hardware block diagram sourced from the Lab Handout. This included an organization into three state registers, which were implemented via three instances of the integrator module, and a series of prior calculations to derive values for dx , dy , and dz , with the effective timestep implemented through distributed right shifts (in this case bit-shifted by 8 to reduce logic).

4.1 Signed Multiply

The first commonly module that was used for this Lorenz Integrator was a signed multiplier for fixed bit calculations provided in the lab handout's "code snippet". This multiplier took two 27 bit inputs and returned a 27 bit output, the maximum multiply operation that can occur with one DSP block on the Altera DE1-SoC. As truncation must occur in this multiplication, an intermediate 54 bit wire was assigned the output of $a * b$, which was then truncated to maintain an equal number of fractional bits and to keep the sign bit to ensure proper signed arithmetic. In the case of our 7.20 two's complement representation,

```
assign out = {mult_out[53], mult_out[45:20]};
```

where `mult_out[53]` is the sign bit of the result and `mult_out[45:20]` represents the numerical portion of the result considering the fixed-point representation. This special multiplication was used to ensure that our Lorenz integrations values would retain proper values after arithmetic. One issue; however, that arose with this multiply was a high sensitivity to overflow that occurred when calculating update values. As covered in the following section.

4.2 Calculation of Update Values

As specified mathematically, the prescaled dx , dy , and dz values were calculated through a series of addition, subtraction, and multiplications, defined by the following equations: $dx = \sigma(y - x)$, $dy = x(\rho - z) - y$, $dz = xy - \beta z$. To perform these equations, a series of adders, subtractors and signed multipliers were to be used, followed by a right shift of 8, denoted by " $>>> 8$ ", which would effectively multiply each value by $1/256$, the scaling value of dt . In practice however, large multiplications, such as a βz or $x(\rho - z)$ multiplication would often result in overflow in the signed multiplier due to the limited dynamic range of the 7.20 fixed point representation. This led to overflow values which were significantly different from expected values and degraded the calculation of the Lorenz system. To fix such issue, a series of synthetic bit shifts for dt scaling were added intermediately in the calculation process of $dxdt$, $dydt$, and $dzdt$, which prevented overflow. These bit-shifted calculations were as follows: $dxdt = \sigma((y - x) >>> 8)$, $dydt = (((x >>> 8)(\rho - z)) - (y >>> 8))$, $dzdt = ((x >>> 8)y - \beta(z >>> 8))$. These new $dxdt$, $dydt$, and $dzdt$ values were now ready to be passed into the accumulation step in the integrator by digital direct analyzer, which will be covered in the next subsection.

4.3 Integration by Digital Direct Analyzer

Each state variable in the system is implemented using a Digital Differential Analyzer (DDA) integrator, which was instantiated as synchronous RTL modules. The DDA integrator used to calculate the Lorenz system performs a first-order Euler integration by accumulating a time-scaled derivative value into a state register sequentially. A block diagram of such system sourced from the Lab Handout is shown below. Such system includes as previously mentioned a state register to store the current value, which gets accumulated with an calculated value, which is multiplied by a corresponding timestep, to which the value is added to the previous register value. At this point, to allow for signal reset, a mux is inserted to allow for the possibility to return to initial conditions in the case of a reset signal. This integrator is instantiated three separate times, once for x , once for y , and once for z values. The values from this system are transmitted to the ARM HPS and additionally routed back for further accumulation. The integrator takes in a initial value, a clock signal, reset signal, and an accumulation function (for example $dxdt$), and operates synchronously.

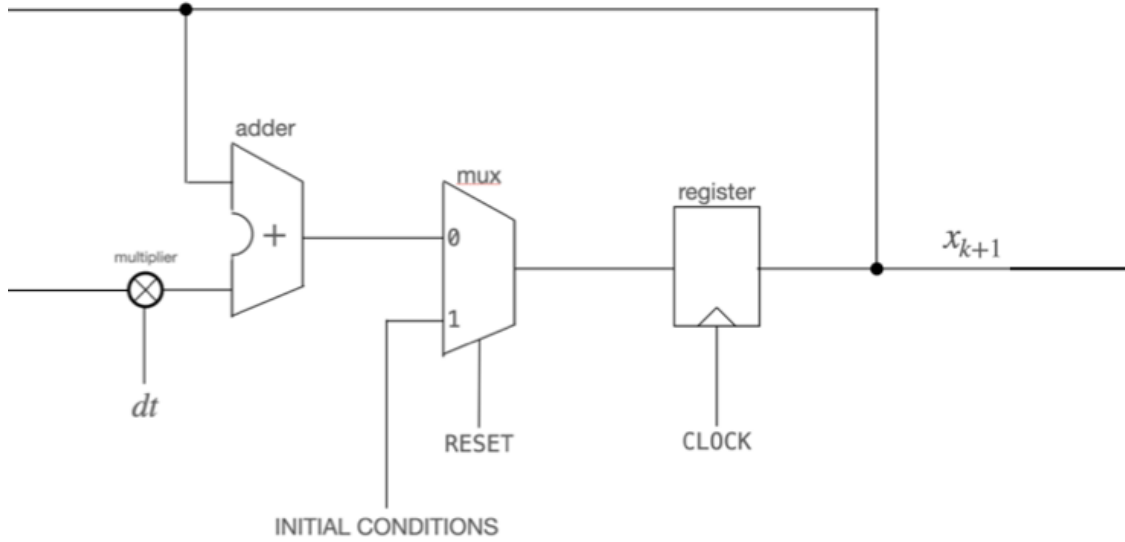


Figure 1: Block diagram of the DDA integrator used to implement Euler integration of the Lorenz system (adapted from [1]).

The RTL for the Lorenz system calculation includes inputs for σ, β, ρ, dt alongside $xk_initial, yk_initial, zk_initial, clk$, and $reset$, and returns a continuous update of xk, yk , and zk . Three parallel operations are occurring in each clock cycle, which includes recalculating $dxdt, dydt$, and $dzdt$, then accumulating the previous values with these new values than, unless $reset$, updating the registers with new values for xk, yk , and zk . Three total integrators are used in this Lorenz integrator, alongside four signed multiply modules, greatly within the scope of DSP and LUT availability on the DE1-SoC.

5 Design and Testing Methods

5.1 Hardware Testing

The primary form of testing our Verilog design was done by comparing the results of our simulation with the output of a Python Lorenz implementation. We then verified that the ModelSim results matched what was actually happening on the FPGA by using SignalTap (see figure 2). A basic testbench was instantiated in Verilog, beginning by setting initial inputs as the following: $dxdt = \sigma((y - x) >>> 8)$, $dydt = (((x >>> 8)(\rho - z)) - (y >>> 8))$, $dzdt = ((x >>> 8)y - \beta(z >>> 8))$. By shifting midway through calculating dx, dy, dz and integrating the dt through this process, we were able to eliminate this overflow leading to the ModelSim and SignalTap plots shown above.

```
dt = (1./256)
x = [-1.]
y = [0.1]
z = [25.]
sigma = 10.0
beta = 8/3
rho = 28.0
```

inside of an instantiated Device Under Test in our non-synthesizable Verilog, where a manual clock was initialized and our system was automatically reset to provide a clean starting point of our initial conditions. Through simulated clock cycles, the Lorenz integrator was able to be verified through the output from ModelSim as shown in 3.

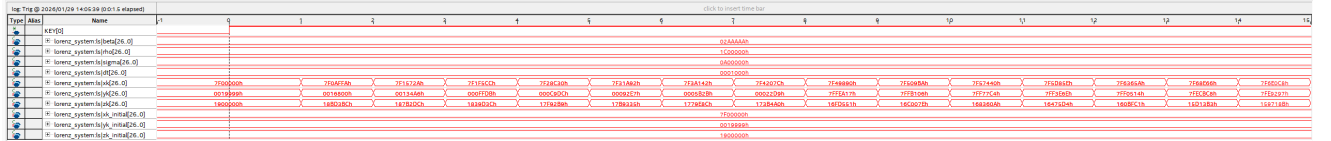


Figure 2: SignalTap Output for Lorenz Integrator

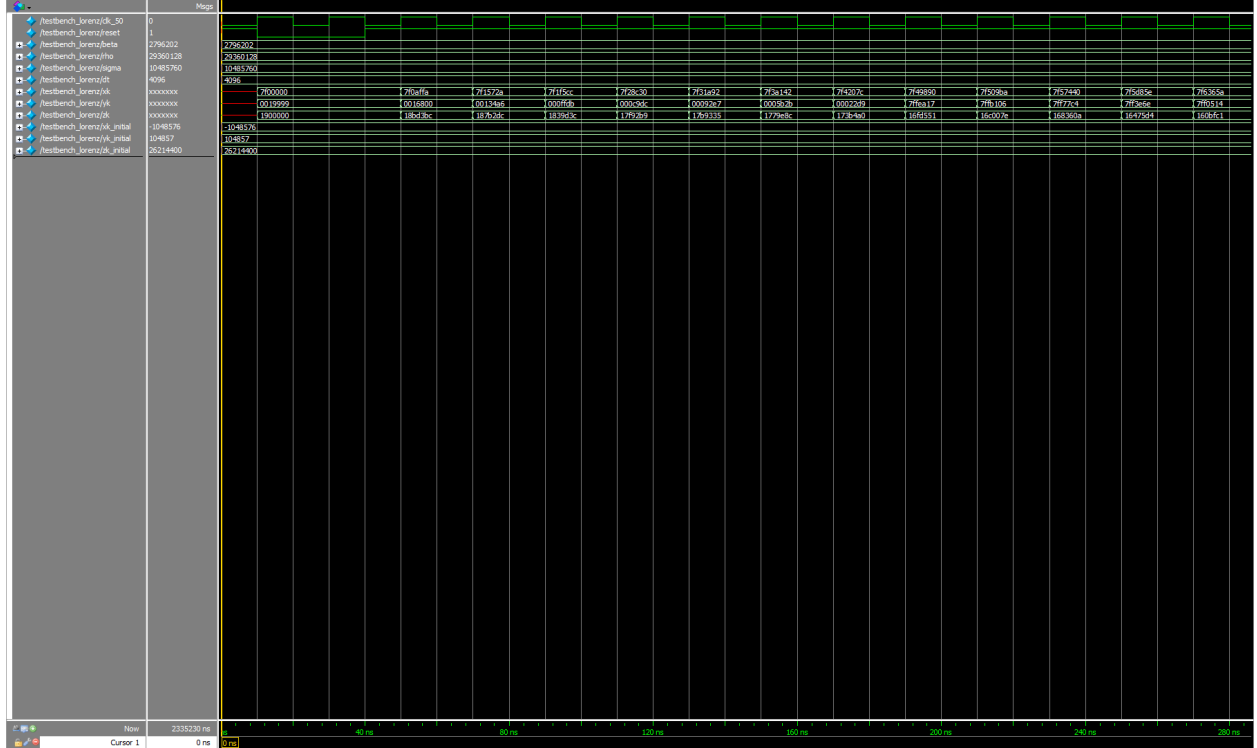


Figure 3: ModelSim Output for Lorenz Integrator

We noticed that the output of the Python integrator matched our implementation for the first few cycles, until it diverged quickly. As discussed in lecture, this is because the Python implementation used floating point math to compute the curve, and our hardware uses fixed-point. This difference in precision results in the initial conditions to be slightly different, something that the Lorenz system is particularly sensitive to. Because the results appeared 'close enough,' combined with the fact that our plots appear correct, as shown in 4, we were confident that our hardware is correct.

This 'close enough' verdict provided by our results is an expected feature of the Lorenz system, where as a chaotic system, small changes in inputs lead to a infinitely different changes in the output. In this such case it is expected that small inconsistencies in fixed vs floating point representations will quickly diverge from one another and through confirmation of an initial set of data points such as the matching of the first set of data points across fixed and floating point, allows us to ensure the correctness of our Lorenz system, as this difference in representation forced a shift from quantitative to qualitative measures as accumulation continued. This testing strategy agreed with the known Lorenz dynamics to ensure proper functionality of our Lorenz integrator

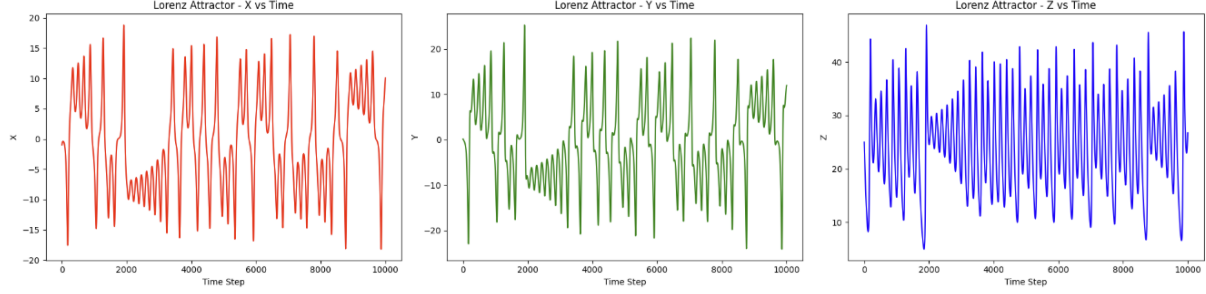


Figure 4: x vs t, y vs t, z vs t Python Plots

As described in the previous section, while testing our implementation initially through ModelSim, we encountered a phenomena where our intregration patterns didn't follow the chaotic action as shown in the python plots but rather followed a large "charging" pattern where after strong initial actions, the values remained fairly constant. We determined that this was a result of overflow in our pre-integration steps of calculating dx , dy , and dz . This led to the change listed above where we

In addition to full-system testing, individual RTL components were verified through closer inspection of our system. It was ensured that not only, plots looked visually correct, but that each module was ensuring correct calculations in hardware. Reset behavior was tested by asserting a reset deterministically in our testbench to ensure that initial values were presented in the event of a reset. After preforming such reset, it was verified that proper integration occurred.

5.2 Software Testing

The software for this assignment was only responsible for taking the outputs from the hardware and drawing it to the screen. To ensure correctness, we had to verify that we were (1) correctly communicating with the FPGA for each parameter and (2) drawing the plot to the screen accurately.

We started by hard coding a C program that grabs the value on each output (x , y , and z) and serving it to the serial output with a simple *printf* call in a loop. We were able to compare this output with the output we saw in both ModelSim and SignalTap to verify that we were seeing the actual output. Once we implemented the interactive loop that allows users to manipulate the parameters, we verified that the HPS was able to modify the outputs (σ , β , ρ , initial conditions) by changing them, resetting the program, and observing the differences in output.

Graphically, we verified our software by allowing it to run continuously for several minutes under different initial conditions, and stress testing the interactive components. Some of these stress tests included maximum speed, minimum speed, high-frequency resets, changing parameters without a reset, and more.

6 Results

7 Chip Planner Layout

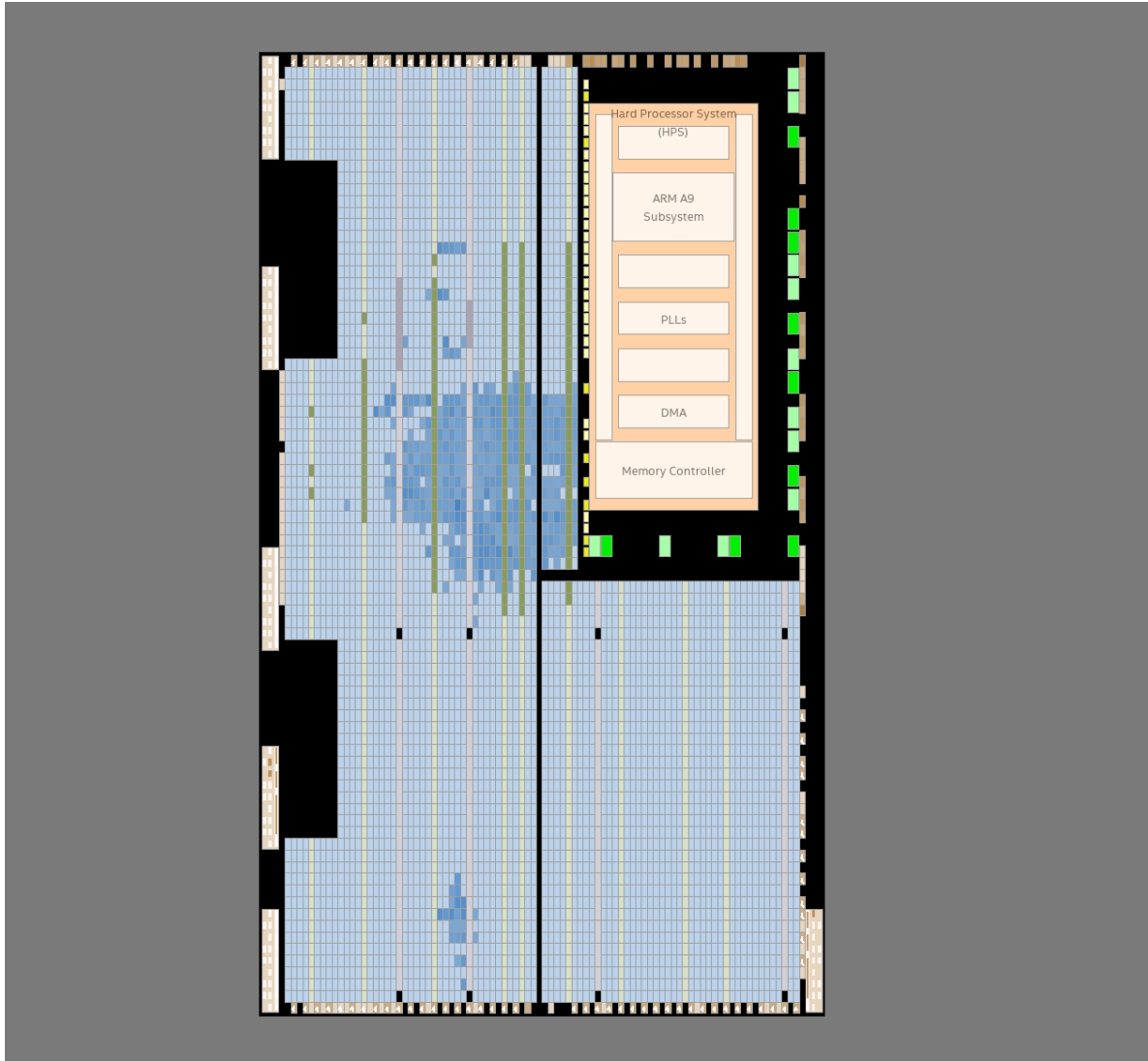


Figure 5: FPGA Chip Planner view showing placement of the system modules and routing resources.

The Chip Planner view provides a physical representation of how the synthesized design is mapped onto the FPGA fabric. Unlike the RTL schematic, which shows logical connectivity, the chip planner shows the *actual hardware implementation* after synthesis, placement, and routing.

Each block in the figure corresponds to a placed module (such as compute units, memory blocks, and interconnect logic), and their locations reflect how the CAD tool distributed logic across the available adaptive logic modules (ALMs), routing channels, and embedded

resources such as block RAMs and DSP units. The routing lines illustrate how signals physically travel between modules on the chip.

This view is significant because FPGA performance is not determined only by the correctness of the Verilog design but also by physical placement and routing. Poor placement can lead to long routing paths, increased propagation delay, and timing violations. By examining the Chip Planner, we can verify that:

- modules are distributed rather than congested in one region,
- memory and compute units are placed near each other to reduce routing delay,
- critical interconnect paths do not span large portions of the chip.

Therefore, the Chip Planner confirms that the design successfully maps to real hardware resources and helps diagnose timing issues, routing congestion, and performance bottlenecks that are not visible at the RTL level.

8 Hardware Resource Utilization

Quartus Prime reports that our synthesized design uses 23,076 ALMs out of 32,070 available (72%), 29,639 registers, 2,382,234 block memory bits out of 4,065,280 (59%), and 15 DSP blocks out of 87 (17%). In addition, the design uses 368 I/O pins out of 457 (81%), 3 PLLs out of 6 (50%), and 1 DLL out of 4 (25%).

These hardware usage results directly correspond to the architecture of our Lorenz attractor visualization system, which consists of a hardware Euler integrator implemented in Verilog and a VGA graphics subsystem controlled by the HPS.

The high ALM (logic element) utilization is primarily caused by the arithmetic datapath in the `lorenz_system` module and the VGA video generation logic. The Lorenz equations require multiple additions, subtractions, and comparisons every clock cycle (e.g., $y - x$, $\rho - z$, and accumulation inside the integrator). In addition, the VGA drawing routines continuously compute pixel coordinates and boundaries for rendering points and trajectories. These operations are implemented using combinational logic and therefore consume a large portion of the FPGA's programmable logic fabric.

The large register count is expected because the system is fully synchronous. Each state variable (x , y , and z) is stored in registers inside the `integrator` module:

Every clock cycle, the new value is written back into the state register. Additional registers are also required for video timing counters, pixel coordinates, and pipeline stages in the arithmetic units. As a result, nearly thirty thousand flip-flops are used.

The DSP block usage comes from the nonlinear terms of the Lorenz equations. In particular, the products xy , xz , and βz are computed using the `signed_mult` module:

```
assign mult_out = a * b;
assign out = {mult_out[53], mult_out[45:20]};
```

Quartus maps these multiplications onto dedicated hardware multipliers (DSP blocks), which are much faster and more area-efficient than implementing multiplication using logic

gates. Since the Lorenz system requires only a few multiplications per iteration, only 15 of the available 87 DSP blocks are used, leaving significant computational headroom.

The block memory usage is mainly associated with the VGA graphics system and HPS communication. The framebuffer and video timing logic require on-chip memory to store pixel information and maintain continuous display output. This explains the moderate BRAM utilization of 59%.

Finally, the PLLs are used to generate multiple clock domains. The system requires separate clocks for the FPGA system logic and the VGA pixel clock, so multiple phase-locked loops are necessary to produce the correct frequencies.

8.1 Critical Path

In our design, the longest combinational logic chain is expected to live inside the Lorenz derivative datapath (the `lorenz_system` module), where multiple fixed-point operations are performed in a single cycle to compute `dxdt`, `dydt`, and `dzdt`. In particular, the `dydt` and `dzdt` expressions contain the deepest arithmetic chains because they involve both a multiplication and additional add/subtract stages:

$$\text{dydt} = ((x \ggg 8)(\rho - z)) - (y \ggg 8), \quad \text{dzdt} = (x \ggg 8)y - \beta(z \ggg 8).$$

These paths include: (1) subtraction to form $(\rho - z)$, (2) at least one 27×27 signed multiply mapped to a DSP block, and (3) one or more add/subtract operations to combine terms. Even when the multiplication itself is implemented in a dedicated DSP block, the surrounding sign-extension, truncation, shifting, and adder logic still contributes to the overall delay, especially when chained.

As a result, we expect the critical path to be a register-to-register path from the current state registers (x_k, y_k, z_k) through the combinational derivative logic (subtract $\rightarrow$ multiply $\rightarrow$ add/subtract $\rightarrow$ shift/truncate) and into the next-state accumulation input of the `integrator` registers.

9 Documentation

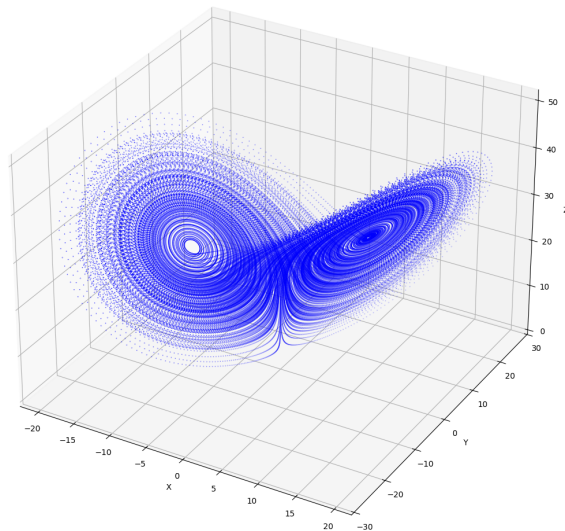


Figure 6: 3D Plot generated from our SignalTap output.

We captured the values for the x , y , and z state variables from the FPGA using SignalTap to plotted to reconstruct the Lorenz attractor. The characteristic butterfly shape confirms that the hardware integrator is producing the correct system dynamics.



Figure 7: VGA Display produced by the HPS on the DE1-SOC.

This was our final demo displayed on the FPGA, demonstrating all the parameters a user

can modify on the serial monitor and the resulting graphs based on those set parameters by the users.

9.1 Program Listings

Our C program uses several macros that may appear cryptic without explanation. To reduce this confusion we have included a detailed listing of the most important macros used in our program. This includes some macros that were not created by us, but instead by Hunter and Bruce in the VGA Color Primitive demo code. We have included these macros in our listing for completion. See Table 1 and Table 2 for these listings.

Table 1: Memory Map and Hardware Address Regions

Region / Symbol	Description
SDRAM_BASE	Physical base address of SDRAM pixel buffer (0xC0000000).
SDRAM_END	End physical address of SDRAM region (0xC3FFFFFF).
SDRAM_SPAN	Total size of SDRAM region (0x04000000).
FPGA_CHAR_BASE	Base address of FPGA character buffer (0xC9000000).
FPGA_CHAR_END	End address of character buffer (0xC9001FFF).
FPGA_CHAR_SPAN	Character buffer size (0x00002000).
HW_REGS_BASE	Base address for HPS lightweight (LW) hardware registers (0xFF200000).
HW_REGS_SPAN	Address span for LW register mapping (0x00005000).
*_OFFSET	Offsets relative to HW_REGS_BASE: • 0x10 — X register • 0x20 — Y register • 0x30 — Z register • 0x40 — σ parameter • 0x50 — β parameter • 0x60 — ρ parameter • 0x70, 0x80, 0x90 — Initial X/Y/Z values • 0x100 — Control register (clock/reset bits)

Table 2: Software Macros and Helper Utilities

Macro / Symbol	Description
SCALE	Fixed-point scale factor 1048576 (2^{20}) used for float fixed conversion.
VGA_PIXEL(x,y,color)	Writes a 16-bit RGB565 pixel to the VGA buffer at (x, y) using $((y)*640 + (x)) \ll 1$ address calculation.
RESET_HIGH	Assert reset (set bit 1 of control register).
RESET_LOW	De-assert reset (clear bit 1).
CLK_HIGH	Set clock bit (bit 0).
CLK_LOW	Clear clock bit (bit 0).
float2fix(a)	$(\text{int})(a \times 2^{20})$
fix2float(a)	$(\text{float})(a)/2^{20}$
int2fix(a)	$a \ll 20$
fix2int(a)	$a \gg 20$

9.2 User Interface

We implemented a simple command line interface (CLI) for changing parameters and adjusting the speed of our solvers. We run our CLI in a separate *pthread* which reads user input from a blocking *scanf* call, and handles it in a case statement. This thread loops indefinitely until the main thread exits (ctrl-C).

The CLI prompts the user to first enter one of several options:

r/b/s/x/y/z/start/stop/reset/[(speed up)](slow down)

The single character parameters stand for Rho, Beta, Sigma, Initial X, Initial Y, Initial Z. The user will enter the corresponding character, then they will be prompted to enter the new value. The change will be reflected immediately in the parameter list on the VGA display. This interface also enables pausing/unpausing, resetting, and adjusting speed. These parameters do not require a second input and will take effect immediately upon hitting enter.

Table 3: Program Listing Table: Mandelbrot Software Control Program

Module / Block	Type	Description
<code>control_thread</code>	Software Thread	Keyboard control thread for interactive Mandelbrot navigation. Uses <code>ncurses</code> to read user key presses and updates the current complex-plane viewing window by changing <code>cr_range</code> , <code>ci_range</code> , <code>cr_start</code> , and <code>ci_start</code> . Supports zooming in/out and panning across the fractal.
<code>main</code>	Software Control / Host Interface	Main host-side control routine. Opens <code>/dev/mem</code> , maps the lightweight HPS-to-FPGA bridge into virtual memory, initializes pointers to memory-mapped FPGA control registers, launches the control thread, and continuously writes updated Mandelbrot view parameters to the FPGA. Also reads back the final cycle count and reports total rendering time in milliseconds.
Fixed-point conversion macros	Software Utility	Macros including <code>float2fix</code> , <code>fix2float</code> , <code>int2fix</code> , <code>fix2int</code> ,

9.3 Verilog

9.3.1 Design

Table 4: Program Listing Table: Verilog Lorenz System Design

Module	Type	Description
<code>testbench_lorenz</code>	Testbench	Top-level simulation harness. Generates <code>clk_50</code> and <code>clk_25</code> , applies reset sequencing, initializes fixed-point parameters and initial conditions, and instantiates <code>lorenz_system</code> as the DUT.
<code>integrator</code>	RTL (Sequential)	Single-state Euler accumulator used as an integrator. On reset, loads <code>InitialOut</code> into the internal register; otherwise updates state via <code>v1new = v1 + funct</code> . Outputs the current state <code>out = v1</code> .
<code>signed_mult</code>	RTL (Combinational)	Signed fixed-point multiplier for 7.20 two's-complement format. Forms the full product <code>mult_out = a*b</code> (54-bit) and truncates/selects bits to return a 27-bit 7.20 result via <code>{mult_out[53], mult_out[45:20]}</code> .
<code>lorenz_system</code>	RTL (Datapath + State)	Implements the Lorenz system Euler update in 7.20 fixed-point. Computes $\dot{x} = \sigma(y-x)$, $\dot{y} = x(\rho-z)-y$, $\dot{z} = xy-\beta z$ using shifts and <code>signed_mult</code> , then integrates <code>xk</code> , <code>yk</code> , <code>zk</code> using three <code>integrator</code> instances.

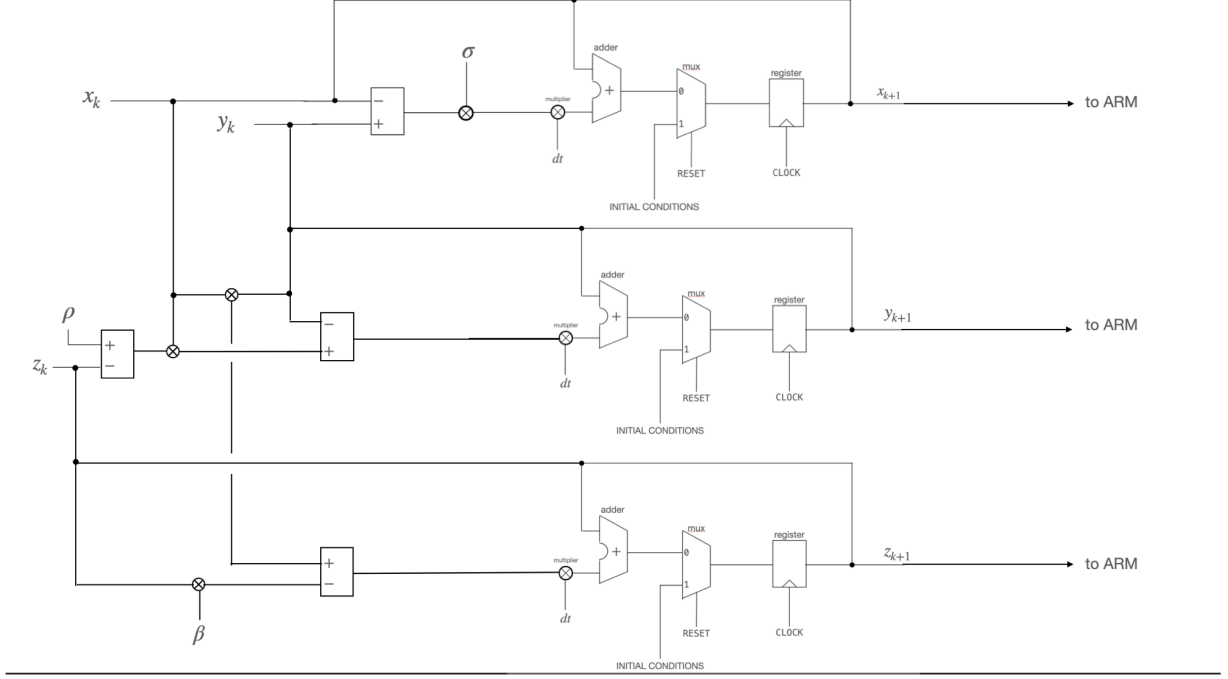


Figure 8: Overall hardware architecture of the Lorenz system implementation (adapted from [1]).

9.3.2 Platform Designer

We used Platform Designer (formerly QSys) to create the PIO connections on the Avalon bus that the HPS and FPGA use to communicate. We added a total of 10 PIO modules to this bus, 7 outputs and 3 inputs relative to the HPS. The directionality and bitwidths of these PIOs is described in Figure 9. The names are self descriptive, except for *HPS_CTRL*. This 8-bit PIO is used to drive the clock/integrator step (*HPS_CTRL*[0]) and *RESET* (*HPS_CTRL*[1]) from the HPS.

The parameters and X , Y , Z are represented in 7.20 fixed point notation on the FPGA side. Our C program has several macros defined for converting between fixed point and floating point for processing and display. These macros are summarized in Table 2. Because this is a 27 bit data type and C integers are 32 bits, values which were passed from the FPGA to the HPS had to be sign-extended to maintain their actual value in two's complement. This is done by pre-pending 5 copies of the MSB of each value on the FPGA side. The notation looks like the following:

```
.lorenz_x_external_connection_export ({5xk[26]}}, xk})
```

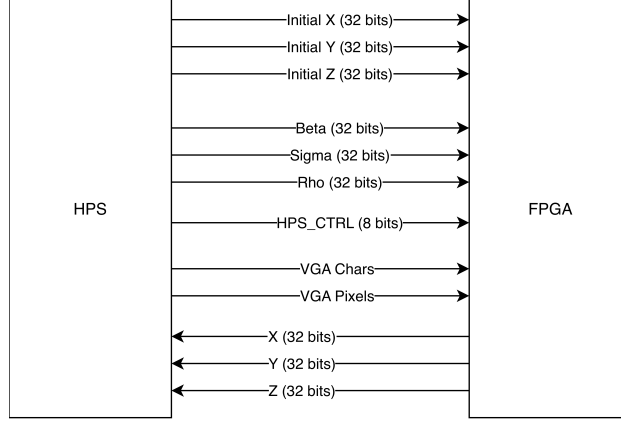


Figure 9: Directionality of PIO communication set in Platform Designer (QSys)

Platform Designer was also configured to allow the HPS to directly write the VGA pixel and char memory on the Avalon bus using the VGA Subsystem IP provided by Altera. This allowed us to draw pixels and characters on the VGA screen from the HPS directly. This was advantageous because it made it simple to interact with the VGA from our C program. In future labs, this method may introduce a memory bottleneck as it used a large chunk of the address space on the Avalon Bus. This also required data to be passed from the FPGA to the HPS, and then from the HPS to the VGA rather than directly from the FPGA to the VGA.

10 Additional Information

10.1 Mathematical Considerations

The specific form of integrator used for this hardware ODE solver applied to the integration of the Lorenz system used first order Euler Integration, a discrete time integrator that follows the equation that $x_{n+1} = x_n + h \cdot f$, where f represents the equations $dx = \sigma(y - x)$, $dy = x(\rho - z) - y$, $dz = xy - \beta z$. However, the Lorenz system is a continuous system, so the Euler integrator is approximating how the system behaves between ticks. We must consider the error accumulated through this method of integration as the real slope changes continuously in the Lorenz system, but we model it with Euler integration as remaining constant for Δt . This leads to both local errors at each time step, and as the Lorenz system is chaotic, also global errors which accumulate over time. The second point of error occurs from the use of 7.20 fixed-point. With 7.20 fixed-point, the smallest value that can be represented would be 2^{-20} or 9.53674316e-7. As a result of this, most real numbers get rounded. This happens not just for updated values, but also initial conditions and parameters. For example, if $\beta = 8/3$, this would need to be approximated to 2.66666603, a difference of 0.000000063, what may look like a small difference, but given the Lorenz system being chaotic, where it exhibits extreme sensitivity based on difference in initial conditions, this difference makes for different calculations as what may be calculated in a floating-point represented simulation. These differences should be interpreted not as errors, but as mathematical differences that can be explained through numeric representation. As a result, it is difficult to verify the

correctness of the calculated Lorenz system through numerical inspection, but correctness should be verified instead through short-term quantitative agreement and by observing the long-term qualitative behavior of the Lorenz attractor.

10.2 How you implemented the DDA circuits:

A Digital Differential Analyzer (DDA) simulates integration via methods such as Euler integration to numerically solve differential equations. Mathematically, this integrator has the form $x_{n+1} = x_n + (dt)(dx)$. The basic DDA is implemented via an adder and state value register, which adds the previous integrated value and a new derivative multiplied by a specific time step. For our Lorenz integrator, we created three of these basic DDAs with an additional mux to reset the integrator if desired which provided the initial conditions to the integrator to restart calculating. Additionally, to calculate dx , dy , and dz , we defined a series of equations, where $dx = \sigma(y - x)$, $dy = x(\rho - z) - y$, $dz = xy - \beta z$. This, theoretically, multiplied by dt , which at $1/256$ was the equivalent of a right bit shift of 8 and then was added to the previous value. Practically; however, as we have limited bits and the DSP modules on the DE1-SoC could do a maximum of 27×27 multiplies, our x , y , and z values were stored in 7.20 fixed point representation. This number system, as with any number system has the potential to overflow. As a result of this overflow, rather than bit shifting after fully calculating dx , dy , and dz , it was imperative to take intermediate steps to ensure that the values were divided before overflowing from large numbers multiplying rather than doing the multiplication first than dividing a possible overflown value. These hardware flows were as follows: $dxdt = \sigma((y - x) \ggg 8)$, $dydt = (((x \ggg 8)(\rho - z)) - (y \ggg 8))$, $dzdt = ((x \ggg 8)y - \beta(z \ggg 8))$.

11 Comments

This lab helped us understand the difference between writing code that looks correct and building hardware that actually works after synthesis. Much of our time was spent debugging rather than implementing the Lorenz equations themselves. Many early issues came from mismatched wire widths, signed vs. unsigned values, and incorrect module connections. In several cases the design compiled but produced incorrect behavior, which required us to inspect waveforms in ModelSim and SignalTap instead of only reading the code.

One of the main challenges was handling overflow and scaling in the 7.20 fixed-point format. The Lorenz equations are straightforward in floating-point software, but hardware multipliers have limited range, so some intermediate calculations became too large. We solved this by shifting values earlier in the datapath instead of only at the end of the computation. This made us better understand how number representation directly affects hardware behavior, especially for a chaotic system like the Lorenz attractor where small numerical changes quickly grow.

Working with the HPS and FPGA together was also an important part of the lab. Debugging communication required verifying that the FPGA produced correct values, the HPS read the correct addresses, and the display interpreted the fixed-point values properly. A mistake in any one of these steps looked similar from the screen output, so we had to test